gemstone-rs BridgeRoot and Object Mapping

A first MagLev-style bridge-root mapping layer over OOPs

Generated from the Markdown sources in gemstone-rs/docs.

Contents

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

The design is connector-inspired, but not transparent persistence. That is an important Rust boundary. The mapping layer should make GemStone objects easier to use from Rust, while still making every remote read, write, transaction, and materialization decision visible in code review.

Which Mapping Layer Should I Use?

Layer	Use it when	What stays explicit
`Oop`	You need the raw GemStone object reference or are writing low-level tooling.	Selector sends, conversion, retain/release, and transaction policy.
`BridgeValue`	You are exploring a payload shape or building UI/CLI inspection tools.	Unsupported objects remain `BridgeValue::Oop`; depth limits are visible.
`BridgeMapped`	You have a stable dictionary-backed Rust payload.	Field names, key types, nested structs, vectors, maps, and option handling.
`#[derive(BridgeMapped)]`	You want normal Rust structs without writing the mapping boilerplate by hand.	Per-field key overrides and symbol/string key policy.
`BridgeRoot`	You need a stable GemStone dictionary root for Rust-owned payloads.	The root name, entry key type, commit/abort behavior, and write scope.
`Remote<T>` / `ObjectRef<T>`	You have an existing OOP and want an explicit cached Rust view of it.	`refresh(&mut Session)`, `set_value`, `save(&mut Session)`, and materialization profile.
Codegen wrappers		

Layer	Use it when	What stays explicit
	You want checked-in Rust wrappers around selectors or mapping configs.	Generated source is reviewed, diffed, checked, and committed.

For most application code, start with `BridgeRoot` plus `BridgeMapped`. Move to `Remote<T>` when the object identity itself matters, for example when a live GemStone object should be refreshed, edited as a Rust value, then explicitly saved back. Use raw `Oop` and `Session::perform` when the object is not yet stable enough to model.

Mapping Rules

The current policy is:

- no hidden network calls from normal Rust field access
- no automatic lazy loading from `Deref` or property access
- no automatic save on `Drop`
- all live operations require `&mut Session`
- all write persistence is explicit: `BridgeRoot::commit`, `BridgeRoot::transaction`, `Remote::save`, or `Session` transaction helpers
- direct `Oop` access remains available at every layer

That gives gemstone-rs a path toward MagLev-style productivity without making Rust code pretend a remote GemStone object is local memory.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::put_mapped_with_key_type`
- `BridgeRoot::put_field`
- `BridgeRoot::put_field_with_key_type`
- `BridgeRoot::put_string`
- `BridgeRoot::put_string_with_key_type`
- `BridgeRoot::put_symbol`
- `BridgeRoot::put_symbol_with_key_type`
- `BridgeRoot::put_smallint`
- `BridgeRoot::put_smallint_with_key_type`

- `BridgeRoot::put_bool`
- `BridgeRoot::put_bool_with_key_type`
- `BridgeRoot::put_vec`
- `BridgeRoot::put_vec_with_key_type`
- `BridgeRoot::put_map`
- `BridgeRoot::put_map_with_key_type`
- `BridgeRoot::put_optional`
- `BridgeRoot::put_optional_with_key_type`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_bridge_value`
- `BridgeRoot::get_bridge_value_with_key_type`
- `BridgeRoot::get_bridge_value_with_depth`
- `BridgeRoot::get_field`
- `BridgeRoot::get_field_with_key_type`
- `BridgeRoot::get_vec`
- `BridgeRoot::get_vec_with_key_type`
- `BridgeRoot::get_map`
- `BridgeRoot::get_map_with_key_type`
- `BridgeRoot::get_optional`
- `BridgeRoot::get_optional_with_key_type`
- `BridgeRoot::get_string`
- `BridgeRoot::get_string_with_key_type`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_smallint_with_key_type`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_bool_with_key_type`
- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_dictionary_with_key_type`
- `BridgeRoot::get_mapped`
- `BridgeRoot::get_mapped_with_key_type`
- `BridgeRoot::remove`
- `BridgeRoot::remove_with_key_type`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeRoot::keys`
- `BridgeRoot::contains_key`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_bridge_value`
- `BridgeDictionary::at_bridge_value_with_key_type`
- `BridgeDictionary::at_bridge_value_with_depth`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`

- `BridgeDictionary::at_bool`
- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_field`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeDictionary::at_map`
- `BridgeDictionary::at_optional`
- `BridgeDictionary::put_field`
- `BridgeDictionary::put_string`
- `BridgeDictionary::put_symbol`
- `BridgeDictionary::put_smallint`
- `BridgeDictionary::put_bool`
- `BridgeDictionary::put_vec`
- `BridgeDictionary::put_map`
- `BridgeDictionary::put_optional`
- `BridgeDictionary::contains_key`
- `BridgeDictionary::keys`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`
- `BridgeMapped`
- `#[derive(BridgeMapped)]`
- `BridgeFieldRead`
- `BridgeFieldWrite`
- `BridgeValue::Nil`
- `BridgeValue::Bool`
- `BridgeValue::SmallInt`
- `BridgeValue::String`
- `BridgeValue::Symbol`
- `BridgeValue::Oop`
- `BridgeValue::Dictionary`
- `BridgeValue::KeyedDictionary`
- `BridgeValue::Array`
- `BridgeValue::from_oop`
- `BridgeValue::from_oop_with_depth`
- `BridgeValue::shape_report`
- `BridgeValueShapeReport`
- `BridgeValueShapeNode`
- `Remote<T>` / `ObjectRef<T>`
- `MaterializationProfile::shallow`
- `MaterializationProfile::deep(max_depth)`
- `DictionaryKeyPolicy::Preserve`
- `DictionaryKeyPolicy::StringOnly`
- `ArrayMaterialization::Materialize`
- `ArrayMaterialization::Reject`

- `IdentityPolicy::PreserveOpaqueOops`
- `IdentityPolicy::ReportRepeatedOops`
- session-local OOP identity ids with `Session::identity_for_oop`

The default root is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict")?;
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
            labels: dictionary.at_map("labels")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
        labels: BTreeMap::from([("source".to_string(), "manual".to_string())]),
    };

    bridge_root.put_mapped("MyTestDict", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.put_string("MyTestStatus", "ready")?;
    bridge_root.put_smallint("MyTestAmount", draft.amount)?;
    bridge_root.put_bool("MyTestApproved", true)?;
```

```
    bridge_root.put_vec("MyTestTags", &["priority".to_string(),
"demo".to_string()])?;
    bridge_root.put_optional("MyTestNote", &Some("front desk".to_string()))?;
    bridge_root.put_map("MyTestLabels", &draft.labels)?;

    assert_eq!(bridge_root.get_string("MyTestStatus")?, "ready");
    assert_eq!(bridge_root.get_smallint("MyTestAmount")?, draft.amount);
    assert!(bridge_root.get_bool("MyTestApproved")?);
    let tags: Vec<String> = bridge_root.get_vec("MyTestTags")?;
    assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
    let note: Option<String> = bridge_root.get_optional("MyTestNote")?;
    assert_eq!(note, Some("front desk".to_string()));
    let labels: BTreeMap<String, String> = bridge_root.get_map("MyTestLabels")?;
    assert_eq!(labels, draft.labels);

    bridge_root.put_map_with_key_type(
        "MyTestLabelsSymbol",
        BridgeKeyType::Symbol,
        &draft.labels,
    )?;
    let symbol_labels: BTreeMap<String, String> =
        bridge_root.get_map_with_key_type("MyTestLabelsSymbol",
BridgeKeyType::Symbol)?;
    assert_eq!(symbol_labels, draft.labels);

    bridge_root.commit()?;
    Ok(())
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}
```

Dictionary Mapping Patterns

Dictionary mapping shows up in three different places. Keep them separate:

Surface	What the key policy controls	Best API
BridgeRoot entry	the key used under `GemStoneRsBridgeRoot`	`put_with_key_type`, `get_*_with_key_type`
Dictionary value	the keys inside the stored GemStone `Dictionary`	`BridgeValue::dictionary`, `BridgeValue::keyed_dictionary`
Rust map field	string-keyed metadata or lookup values	`BTreeMap<String, T>`, `put_map`, `at_map`

Use `BTreeMap<String, T>` when the dictionary is JSON-like metadata. It stores and reads back string-keyed GemStone dictionary entries:

```
use gemstone_rs::{Config, Session};
use std::collections::BTreeMap;

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let labels = BTreeMap::from([
    ("channel".to_string(), "web".to_string()),
    ("priority".to_string(), "high".to_string()),
]);

bridge_root.put_map("BookingLabels", &labels)?;
let loaded: BTreeMap<String, String> = bridge_root.get_map("BookingLabels")?;
assert_eq!(loaded["channel"], "web");
```

Use `BridgeValue::keyed_dictionary` when the dictionary entries themselves must use Smalltalk symbols or a deliberate mix of string and symbol keys:

```
use gemstone_rs::{BridgeKey, BridgeKeyType, BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::keyed_dictionary([
    (BridgeKey::symbol("status"), BridgeValue::from("ready")),
    (BridgeKey::symbol("amount"), BridgeValue::from(100_i64)),
    (BridgeKey::string("externalId"), BridgeValue::from("web-42")),
]);

bridge_root.put("SmalltalkBooking", payload)?;
```

```
let dynamic = bridge_root.get_bridge_value("SmalltalkBooking")?;
assert!(matches!(dynamic, BridgeValue::KeyedDictionary(_)));

let mut dictionary = bridge_root.get_dictionary("SmalltalkBooking")?;
assert_eq!(
    dictionary.at_string_with_key_type("status", BridgeKeyType::Symbol)?,
    "ready"
);
assert_eq!(
    dictionary.at_smallint_with_key_type("amount", BridgeKeyType::Symbol)?,
    100
);
assert_eq!(dictionary.at_string("externalId")?, "web-42");
```

`BridgeValue::from_oop` reads a dictionary with only string keys as `BridgeValue::Dictionary`. If any key is a symbol, it preserves the per-entry policy as `BridgeValue::KeyedDictionary`. That makes explorer output and shape reports honest about what Smalltalk code will see.

One subtle point: `put_map_with_key_type("BookingLabels", BridgeKeyType::Symbol, &labels)` changes the key used to store `BookingLabels` in the containing dictionary. It does not make the entries inside `labels` symbol-keyed. To control entry keys inside the value, build a `BridgeValue::keyed_dictionary`.

Dynamic BridgeValue Inspection

When you do not want a typed struct yet, read a BridgeRoot value back as a dynamic `BridgeValue` tree:

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let payload = BridgeValue::dictionary([
        (
            "customer".to_string(),
            BridgeValue::dictionary([
                ("name".to_string(), BridgeValue::from("Tariq")),
                ("vip".to_string(), BridgeValue::from(true)),
            ]),
        ),
        (
            "items".to_string(),
            BridgeValue::array([
                BridgeValue::dictionary([
                    ("sku".to_string(), BridgeValue::from("A-1")),
                    ("quantity".to_string(), BridgeValue::from(2_i64)),
                ]),
```

```
            ]),
        ),
        ("state".to_string(), BridgeValue::Symbol("ready".to_string())),
        ("note".to_string(), BridgeValue::Nil),
    ]);

    bridge_root.put("BridgeValueInspection", payload.clone())?;
    let dynamic = bridge_root.get_bridge_value("BridgeValueInspection")?;
    assert_eq!(dynamic, payload);
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example bridge_value_inspection
```

Expected output includes:

```
dynamic BridgeValue: Dictionary({"customer": Dictionary(...), "items": Array(...),
"note": Nil, "state": Symbol("ready")})
bridge root identity: <number>
bridge root key count: <number>
```

`BridgeValue::from_oop_with_depth(session, oop, max_depth)` is also available for inspectors and tools. It reads nil, booleans, small integers, characters, strings, symbols, arrays, and string/symbol-keyed dictionaries. When the reader hits an unsupported object, a repeated object, or the depth limit, it returns `BridgeValue::Oop(oop)` instead of pretending the object is transparent Rust state.

For a compact relationship-oriented view, ask the value for a shape report:

```
let report = dynamic.shape_report();
println!("nodes: {}", report.total_nodes);
for node in report.nodes {
    println!("{} {} children={}", node.path, node.kind, node.child_count);
}
```

The CLI form is:

```
gemstone-rs bridge shape BookingDraft --depth 4
```

This reports paths such as `value.customer.#name`, `value.items[1].sku`, node kinds, key policy, child counts, opaque OOPs, report-local identity ids, repeated opaque references, and nil nodes. It is meant for relationship mapping review before generating typed wrappers.

When the same opaque OOP appears more than once, the report also includes an identity group. That gives the CLI, explorer, and VS Code webview a stable way to show relationship paths such as `value.items[2]` and `value.items[3]` both pointing at the same GemStone object.

You can turn the inspected shape into a starter codegen config before writing a typed struct by hand:

```
use gemstone_rs::{codegen, BridgeValue};

let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
]);

let config = codegen::mapping_config_from_bridge_value("BookingDraft", &payload);
println!("{config}");
```

Run the offline example:

```
cargo run -p gemstone-rs --example bridge_mapping_preview
```

Or infer from a live BridgeRoot value:

```
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
```

The output is intentionally reviewable text, not automatic persistence magic. Opaque OOPs, nil fields, symbols, empty arrays, and mixed arrays receive `doc=` notes so you can choose a narrower type before running codegen.

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
```

```
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        amount: 100,
        customer: CustomerDraft {
            name: "Tariq".to_string(),
        },
        tags: vec!["priority".to_string(), "demo".to_string()],
        labels: BTreeMap::from([("source".to_string(), "derive".to_string())]),
        note: None,
    };

    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }
bridge root identity: <number>
```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
`String`	`String`	`BridgeFieldRead` / `at_string`
`i64`	`SmallInteger`	`BridgeFieldRead` / `at_smallint`

Rust field	GemStone storage	Read-back helper
`bool`	`true` / `false`	`BridgeFieldRead` / `at_bool`
`Oop`	raw object reference	`BridgeFieldRead` / `at_oop`
another `BridgeMapped` struct	nested `Dictionary`	`BridgeDictionary::at_mapped`
`Vec<T>`	`Array`	`BridgeDictionary::at_vec`
`BTreeMap<String, T>`	string-keyed `Dictionary`	`BridgeDictionary::at_map`
`Option<T>`	value, `nil`, or missing key	`BridgeFieldRead`

This lets a Rust payload keep normal nested shape:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

Optional fields are useful when a BridgeRoot dictionary evolves over time. `None` writes as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`. `Some(value)` reads through the inner field type, so `Option<CustomerDraft>` still reports nested mapping errors with the full field path.

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234
field booking.customer.name expected GemStone value type String, got OOP 1234
field tags[2] expected GemStone value type String, got OOP 1234
field labels["source"] expected GemStone value type String, got OOP 1234
```

Nested mapped read-back preserves the full field path, and array read-back reports the failing element index. Both are important when generated mappings read back nested payloads from a live stone. Lookup failures are also wrapped with the current path, so missing keys and invalid nested arrays point at `booking.items` or `booking.items[2]` instead of only returning a generic GemStone lookup error.

Dynamic `BridgeValue` read-back uses the same path-aware machinery when called through `BridgeRoot::get_bridge_value` or `BridgeDictionary::at_bridge_value`. That makes it useful for explorer panels and generated-wrapper debugging before you settle on a typed `BridgeMapped` struct.

Key Policy

GemStone dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;

bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored")?;
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

The typed helpers have the same key-policy variants:

```
bridge_root.put_field_with_key_type("BookingLabels", BridgeKeyType::Symbol,
&draft.labels)?;
let labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("BookingLabels", BridgeKeyType::Symbol)?;
```

Those variants choose the lookup key in the containing dictionary. For symbol-keyed entries inside the dictionary value itself, use the dictionary mapping pattern above.

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Remote Object Handles

`Remote<T>` is the Rust-native proxy layer. It is deliberately explicit:

- it stores an `Oop`
- it stores type metadata such as `UserGlobals:BookingDraft`
- normal field access never talks to GemStone
- `refresh(&mut Session)` reads the GemStone object into a cached Rust value
- `set_value(value)` marks the cached value dirty
- `save(&mut Session)` writes the cached mapped value back to the same GemStone

  dictionary object

```
use gemstone_rs::{BridgeMapped, Config, MaterializationProfile, Remote, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    status: String,
    amount: i64,
    labels: BTreeMap<String, String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let initial = BookingDraft {
        status: "draft".to_string(),
        amount: 100,
        labels: BTreeMap::from([("source".to_string(), "remote-
example".to_string())]),
    };

    let oop = {
        let mut bridge_root = session.bridge_root()?;
        bridge_root.put_mapped("RemoteBookingDraft", &initial)?;
        bridge_root.get_oop("RemoteBookingDraft")?
    };

    let mut remote = Remote::<BookingDraft>::with_type(oop,
"UserGlobals:BookingDraft")
        .with_profile(MaterializationProfile::deep(4));

    let loaded = remote.refresh(&mut session)?.clone();
    assert_eq!(loaded, initial);

    let mut updated = loaded;
    updated.status = "confirmed".to_string();
    remote.set_value(updated);
    assert!(remote.is_dirty());
    remote.save(&mut session)?;
    assert!(!remote.is_dirty());

    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example remote_object_mapping
```

Expected output includes:

```
remote loaded: BookingDraft { status: "draft", amount: 100, labels: {"source":
"remote-example"} }
```

```
remote saved: BookingDraft { status: "confirmed", amount: 100, labels: {"source":
"remote-example"} }
```

`ObjectRef<T>` is an alias for `Remote<T>` when that name reads better in application code.

This is not transparent persistence. The handle does not save on drop, does not fetch fields lazily, and does not hide network I/O behind normal Rust access. Every GemStone operation still takes `&mut Session`.

Materialization Profiles

Materialization profiles describe how far a remote value should be read and how strictly the resulting `BridgeValue` tree should be validated:

```
use gemstone_rs::{
    ArrayMaterialization, DictionaryKeyPolicy, IdentityPolicy,
MaterializationProfile,
};

let profile = MaterializationProfile::deep(4)
    .with_dictionary_key_policy(DictionaryKeyPolicy::Preserve)
    .with_array_materialization(ArrayMaterialization::Materialize)
    .with_identity_policy(IdentityPolicy::ReportRepeatedOops);
```

Useful profiles:

Profile	Behavior
`MaterializationProfile::shallow()`	Keep non-immediate remote objects opaque.
`MaterializationProfile::deep(4)`	Read supported dictionaries and arrays up to depth 4.
`DictionaryKeyPolicy::StringOnly`	Reject symbol-keyed dictionaries when a JSON-like shape is required.
`ArrayMaterialization::Reject`	Fail fast if a mapping path unexpectedly contains an array.
`IdentityPolicy::ReportRepeatedOops`	Keep shape reports useful for repeated opaque OOP references.

The profile feeds `Remote<T>::materialize`, `Remote<T>::shape_report`, and tool surfaces such as the explorer and VS Code webview. It does not make remote objects transparent; it makes the read policy explicit and reviewable.

Connector-Style Config

For BridgeRoot dictionaries, `field = ... | type=... | key=...` is enough. For remote GemStone classes, codegen configs can also carry connector-style selector metadata:

```
mapped = Booking
class = UserGlobals:OkzBooking
field = Booking.status | selector=status | return=Symbol
field = Booking.customer | selector=customer | return=Mapped<Customer>
```

For a concrete checked-in example, see:

```
examples/codegen/connector-mapping.codegen
```

It combines `class = UserGlobals:OkzBooking`, selector metadata, return metadata, relationship fields, and dictionary fallback keys. Run `gemstone-rs codegen explain examples/codegen/connector-mapping.codegen` to see the mapping summary without writing generated files.

This lets one config describe both sides of the bridge:

- `class = UserGlobals:OkzBooking` records the GemStone class to inspect or wrap
- `selector=status` records how the remote object is read
- `return=Symbol` records the Smalltalk-facing return type
- `return=Mapped<Customer>` records a relationship to another mapped Rust type

The initial implementation parses and reports this connector metadata through `codegen explain` and `codegen explain --json`. Generation still stays conservative: dictionary-backed `BridgeMapped` code is emitted as before, while remote selector wrappers can be layered on top in a later generator pass.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("BookingDraft")?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft")?;
let identity = bridge_root.identity_id();
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Relationship-Shaped Payloads

Use nested structs and vectors when the Rust boundary needs a relationship-like shape but you still want explicit persistence. This keeps the GemStone side a plain dictionary graph and keeps the Rust side strongly typed:

```
use gemstone_rs::BridgeMapped;
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    name: String,
    email: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct LineItemDraft {
    sku: String,
    quantity: i64,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    items: Vec<LineItemDraft>,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

A failed nested read reports the path that a user would naturally inspect:

```
booking.items[2].customer.name expected GemStone value type String, got OOP 1234
```

This is the practical middle ground between raw OOPs and transparent object persistence: relationships are visible in config and generated Rust source, and the low-level OOP API remains available for special cases.

Comparison With MagLev/GBS

The classic MagLev branch session example uses `userGlobals` directly:

```
session userGlobals at: #MyTestDict put: dict.
session commit.
session disconnect.
```

The closest `gemstone-rs` equivalent is:

```
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
]);

let payload_oop = payload.to_oop(&mut session)?;
session.global_put("MyTestDict", payload_oop)?;
session.commit()?;

let stored = session.global_get("MyTestDict")?;
let mut loaded = BridgeDictionary::from_oop(&mut session, stored);

assert_eq!(loaded.at_string("name")?, "Tariq");
assert_eq!(loaded.at_smallint("amount")?, 100);
assert_eq!(loaded.at_string("currency")?, "GBP");

session.logout()?;
```

Run the checked-in example:

```
cargo run -p gemstone-rs --example maglev_classic_session
```

The MagLev-oriented example uses `bridgeRoot`:

```
session bridgeRoot at: #MyTestDict put: payload.
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;
bridge_root.put_with_key_type("MyTestDict", BridgeKeyType::Symbol, payload)?;
bridge_root.commit_with_retry(0)?;

let mut loaded =
    bridge_root.get_dictionary_with_key_type("MyTestDict", BridgeKeyType::Symbol)?;

assert_eq!(loaded.at_string("name")?, "Tariq");
assert_eq!(loaded.at_smallint("amount")?, 100);
assert_eq!(loaded.at_string("currency")?, "GBP");

session.logout()?;
```

Both examples retrieve the same GemStone-side shape:

```
MyTestDict => {
  name: "Tariq",
  amount: 100,
  currency: "GBP"
}
```

Run the checked-in example:

```
cargo run -p gemstone-rs --example maglev_bridge_root_session
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels
field = BookingDraft.note | type=Option<String> | key=note
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

For quick CLI inspection of the bridge root itself:

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge value BookingDraft --depth 4
gemstone-rs bridge shape BookingDraft --depth 4
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
gemstone-rs bridge sample-config BookingDraft
```

Use `--root OtherBridgeRoot` when you intentionally use a non-default root dictionary, and `--key-type String|Symbol` when you want the key policy to be spelled out in scripts. Equals-style options such as `--root=OtherBridgeRoot`, `--key-type=Symbol`, and `--type=SmallInt` are accepted for CI scripts. The generic `bridge put --type String|Symbol|SmallInt|Bool` form is still available when the value type comes from script data.

`bridge keys` lists each root key with its key OOP, class OOP, `printString`, and session-local identity id.

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The write endpoints require `gemstone-rs-explorer --allow-write`. Without that flag they return HTTP 403, matching the explorer's read-only default.

The VS Code workbench adds the same object-mapping workflow:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oop` remains visible and available.
- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.